

7. Test Environment Setup

7.1 Introduction

A Test Environment is a setup of hardware, software, tools, configurations, test data, and network settings required to execute and validate application functionality. A properly configured test environment ensures that test execution is stable, reliable, and produces consistent results.

For this project, the test environment was configured to support Selenium WebDriver automation, TestNG execution, Maven build management, Git version control, and GitHub Actions CI/CD integration for the Parabank Banking Application.

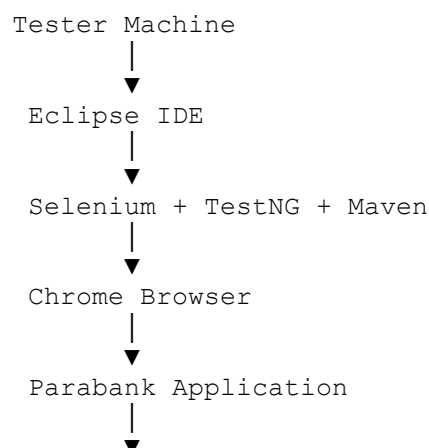
The purpose of this document is to provide a detailed description of all environment components required for successful test automation execution.

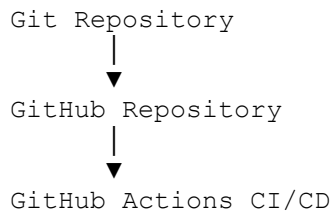
7.2 Test Environment Objective

The objectives of setting up the test environment are:

- Enable automation test execution.
 - Ensure browser compatibility.
 - Support Selenium WebDriver integration.
 - Enable data-driven testing.
 - Support CI/CD pipeline execution.
 - Generate execution reports.
 - Maintain a stable and repeatable testing environment.
-

7.3 Test Environment Architecture





7.4 Hardware Configuration

The following hardware configuration was used:

Component	Specification
Processor	Intel Core i5 or Higher
RAM	8 GB Minimum
Storage	20 GB Free Space
Internet	Stable Broadband Connection

7.5 Software Configuration

Software	Version
Windows OS	Windows 10/11
Java JDK	JDK 21
Eclipse IDE	Latest Version
Maven	Latest Version
Git	Latest Version
Chrome Browser	Latest Version
ChromeDriver	Compatible Version
GitHub Account	Active Account

7.6 Application Under Test (AUT)

Application Name

Parabank Banking Application

Application URL

[Parabank Demo Application](#)

Application Type

Web-Based Banking Application

Functional Modules Tested

- Registration
 - Login
 - Account Overview
 - Open New Account
 - Transfer Funds
 - Bill Payment
 - Find Transactions
 - Request Loan
 - Update Contact Information
-

7.7 Java Installation Setup

Java is required to develop and execute Selenium automation scripts.

Installation Steps

1. Download JDK 21.
2. Install Java.
3. Configure JAVA_HOME environment variable.
4. Add Java bin folder to PATH.
5. Verify installation:

```
java -version
```

Expected Output:

```
java version "21"
```

7.8 Maven Installation Setup

Maven is used for dependency management and project build execution.

Installation Steps

1. Download Maven.
2. Extract Maven folder.
3. Configure MAVEN_HOME.
4. Add Maven bin directory to PATH.
5. Verify installation.

```
mvn -version
```

Expected Output:

Apache Maven 3.x.x

7.9 Eclipse IDE Setup

Eclipse IDE was used for automation development.

Configuration Activities

- Install Eclipse IDE.
- Configure Java project.
- Import Maven project.
- Update Maven dependencies.
- Configure TestNG plugin.

Benefits:

- Easy coding.
 - Debugging support.
 - Maven integration.
 - Git integration.
-

7.10 Selenium Setup

Selenium WebDriver was configured through Maven dependencies.

Purpose

- Browser automation.
- Web element interaction.
- UI validation.

Activities Performed

- Added Selenium dependencies.
 - Configured WebDriver initialization.
 - Created Page Object classes.
-

7.11 TestNG Setup

TestNG framework was used for test execution.

Features Used

- Test execution
- Data Providers
- Assertions
- Reporting
- Listeners

Configuration

Test execution controlled using:

```
testng.xml
```

7.12 Git Setup

Git was configured for version control.

Activities Performed

- Installed Git.
- Initialized Git repository.
- Configured Git username.
- Configured Git email.
- Connected local repository to GitHub.

Verification:

```
git --version
```

7.13 GitHub Setup

GitHub repository was created to store project source code.

Activities Performed

- Created GitHub account.
- Created repository.
- Connected local repository.
- Pushed source code.

Repository includes:

- Automation framework
- CI/CD workflow
- Documentation

- Reports
-

7.14 Test Data Setup

Data-driven testing was implemented using Excel files.

Files Used

- LoginData.xlsx
- TransferFundsData.xlsx
- BillPayData.xlsx

Purpose

- Store test data externally.
 - Improve maintainability.
 - Increase test coverage.
-

7.15 Framework Configuration

Configuration values stored in:

```
config.properties
```

Parameters:

```
browser=chrome  
url=https://parabank.parasoft.com/parabank/index.htm
```

Benefits:

- Easy configuration management.
 - No code modification required.
-

7.16 Report Configuration

Reports generated after execution:

TestNG Reports

- index.html

- `emailable-report.html`
- `testng-results.xml`

Screenshots

Captured automatically on failure.

Execution Logs

Stored inside:

`logs/execution.log`

7.17 Environment Validation

The following validations were performed:

- ✓ Java Installation Verified
 - ✓ Maven Installation Verified
 - ✓ Eclipse Configuration Verified
 - ✓ Selenium Dependencies Verified
 - ✓ Git Configuration Verified
 - ✓ GitHub Repository Verified
 - ✓ Chrome Browser Compatibility Verified
 - ✓ TestNG Execution Verified
-

7.18 Conclusion

The test environment was successfully configured to support automation testing and CI/CD integration. All required software, tools, configurations, and dependencies were installed and verified. The environment provided a stable platform for executing Selenium automation scripts and integrating automated testing into the GitHub Actions pipeline.
